

Продвинутая отладка: анализ дампов памяти с WinDbg и SOS

C# Developer. Advanced



Проверить, идет ли запись

**Меня хорошо видно
& слышно?**





Евгений Волобуев

- ❖ Ведущий програмист (SimbirSoft);
- ❖ Более 6 лет работы в сфере коммерческой и промышленной разработки
- ❖ Опыт разработки высоконагруженных и отказоустойчивых систем;
- ❖ В составе команды создаём мобильный интернет банк для стран латинской америки;



@e_volobuev

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Маршрут вебинара

Тестирование

Что такое WinDBG

Отличие от других инструментов

Полезные команды

Командное соревнование

Разбор результатов

Вопросы и ответы



Цели вебинара

К концу занятия вы сможете

1. Разбираться в инструментах для поиска и устранения ошибок в системе
2. Понимать, где искать причины падения приложения
3. Диагностировать проблемы с памятью
4. Объяснить на собеседовании алгоритм действий выяснения причин падения приложения, помимо просмотра логов



Смысл

Это нужно уметь чтобы:

1. Анализировать дампы из production

2. Понимать .net на уровне «как оно работает внутри»

3. Экономить время на расследовании инцидентов

4. Диагностировать «невидимые» проблемы

5. Проходить собеседования (пересмотры зп на текущей работе)



Тестирование



Что такое memory dump ?

Дамп приложения (memory dump) — это файл, который содержит снимок состояния работающего приложения в определенный момент времени. Простыми словами, это как фотография внутреннего состояния программы: что именно происходило внутри в момент создания дампа.



Дамп приложения включает в себя:

1. Стек вызовов всех потоков
2. Содержимое управляемой и нативной памяти
3. Информацию о загруженных модулях (.dll, .exe, .net сборки)
4. Значения регистров процессора в момент снятия дампа



Зачем нужен дамп?

1. Анализировать зависания (deadlock, бесконечные циклы) без остановки приложения
2. Исследовать высокое потребление памяти (утечки)
3. Диагностировать внезапные падения (crash) в production-среде, где невозможно воспроизвести проблему локально
4. Анализировать состояние приложения без необходимости иметь доступ к работающему процессу



Что такое WinDBG и SOS



WinDBG

— это многоцелевой отладчик для операционной системы Microsoft Windows. Его используют для поиска и устранения ошибок в системе, а также для глубокого анализа работы программного обеспечения.

- Позволяет отлаживать приложения в режиме пользователя, драйверы устройств и саму операционную систему в режиме ядра.
- Используется для анализа дампов памяти, поиска причин «синих экранов» и диагностики сложных крашей.
- Поддерживает расширенные скриптовые и автоматизационные возможности.
- Совместим с другими инструментами: IDA, Radare2 используют его протокол отладки.



Альтернативы

Visual Studio Debugger

Преимущества:

- Интуитивный графический интерфейс с пошаговой отладкой
- Автоматическое отображение локальных переменных, watch windows
- Intellisense для выражений
- Удобная работа с breakpoints
- Отличная интеграция с source code

Недостатки:

- Требуется установки (5-20 GB) - неприемлемо для production серверов
- Не может открывать dump файлы без установленного runtime той же версии
- Ограниченные возможности low-level анализа (heap, handles, locks)
- Медленный на больших дампах



Альтернативы

dnSpy

Преимущества:

- Бесплатный и xcopy deployable (не требует установки)
- Интерфейс как в Visual Studio с пошаговой отладкой
- Автоматическая декомпиляция кода на лету - можно отлаживать без исходников
- Можно редактировать код прямо во время отладки и пересобирать
- Отличен для быстрой отладки на production машинах

Недостатки:

- Проект заархивирован (не развивается)
- Не поддерживает анализ дампов так глубоко, как WinDBG
- Нет команд для анализа heap, GC roots, threadpool state



Альтернативы

JetBrains dotMemory

Преимущества:

- Дружелюбный GUI для анализа памяти
- Отличная визуализация утечек памяти и retention paths
- Автоматическое определение проблемных объектов
- Не требует знания команд - всё через клики

Недостатки:

- Платная (\$299-\$499)
- Только для анализа памяти, не для дедлоков или CPU issues
- Не даёт full control как WinDBG
- Не покажет детали вроде имён статических переменных

Альтернативы

JetBrains dotTrace

Преимущества:

- Performance profiling с визуализацией call tree
- Timeline mode для анализа CPU и memory allocation
- Сравнение двух профилей для измерения оптимизаций
- Интеграция с CI/CD для мониторинга деградации

Недостатки:

- Платная (\$299-\$499)
- Фокус только на performance, не на crashes/hangs



Альтернативы

DebugDiag

Преимущества:

- Автоматический сбор дампов по правилам (crash, hang, high CPU, memory leak)
- Автоматический анализ с генерацией HTML-отчета
- Встроенный memory leak detector для native code
- Бесплатный от Microsoft

Недостатки:

- Автоанализ поверхностный - для глубокого анализа всё равно нужен WinDBG
- Слабая поддержка managed code по сравнению с WinDBG
- Невозможность выполнять произвольные команды

Альтернативы

PerfView

Преимущества:

- Отличный для CPU profiling через ETW трассировку
- GC heap analysis и allocation tracking
- Бесплатный от Microsoft
- Call tree visualization

Недостатки:

- Только для performance, не для crashes/hangs
- Требуется понимания ETW и sampling

Преимущества WinDBG

1. Универсальность

WinDBG - единственный инструмент, который умеет всё:

- Post-mortem debugging дампов
- Live debugging на production
- Kernel debugging
- Mixed mode (managed + native)
- Memory leaks, deadlocks, high CPU, crashes, hangs

2. Глубина анализа

Позволяет копать на любую глубину:

- Heap analysis (`!dumpheap`, `!gcroot`)
- Lock analysis (`!syncblk`)
- Thread pool state (`!threadpool`)
- Handle leaks
- Memory corruption
- Finalizer queue
- Assembly loading issues

3. Production-friendly

- Минимальный размер (~50 MB)
- Не требует тяжелой установки
- Можно работать с любым дампом на любой машине
- Не нагружает систему

4. Автоматизация

Можно писать скрипты для автоматического анализа

5. Полный контроль

Можете выполнить любую команду и получить любую информацию из памяти:

- Читать raw memory
- Анализировать data structures
- Проверять assembly metadata
- Инспектировать JIT code



Альтернативы

Сценарий	Лучший инструмент	Причина
Production crash dump	WinDBG	Lightweight, полный контроль
Memory leak поиск	dotMemory	Быстрее найти с GUI
Performance profiling	dotTrace/PerfView	Специализированные для этого
Deadlock в production	WinDBG	Только он даст full thread analysis
Development debugging	Visual Studio	Удобство и скорость
Debugging без source code	dnSpy	Автоматическая декомпиляция
Автосбор дампов	DebugDiag	Rules-based collection
Kernel debugging	WinDBG	Единственный вариант



Команды

Работа с потоками

Команда	Описание
<code>!threads</code>	Список всех управляемых потоков с состоянием и Lock Count
<code>~Xs</code>	Переключиться на поток X (например, <code>~4s</code>)
<code>~*e !clrstack</code>	Показать стеки всех потоков одновременно
<code>!clrstack</code>	Стек вызовов текущего потока
<code>!clrstack -a</code>	Стек с аргументами и локальными переменными

Команды

Обнаружение дедлоков

Команда	Описание
<code>!syncblk</code>	Все активные блокировки (MonitorHeld=3 указывает на дедлок)

Команды

Анализ памяти

Команда	Описание
<code>!dumpheap -stat</code>	Статистика объектов в куче (топ по размеру)
<code>!dumpheap -type <Type></code>	Все экземпляры указанного типа с адресами
<code>!dumpheap -min <size></code>	Объекты размером больше указанного
<code>!gcroot <address></code>	Цепочка ссылок от корня GC до объекта (причина утечки)
<code>!do <address></code>	Содержимое объекта по адресу
<code>!objsize <address></code>	Реальный размер объекта включая ссылаемые объекты

Команды

Анализ производительности

Команда	Описание
<code>!threadpool</code>	Состояние ThreadPool (потoki, очередь задач, CPU)
<code>!runaway</code>	Потоки с наибольшим временем CPU
<code>!finalizequeue</code>	Очередь объектов, ожидающих финализации

Анализ исключений

Команда	Описание
<code>!dumpheap -type System.Exception</code>	Все объекты-исключения в памяти
<code>!pe <address></code>	Форматированный вывод исключения (StackTrace, Message)

Команды

Автоматический анализ

Команда	Описание
<code>!analyze -v</code>	Автоматический анализ дампа с предположением причины
<code>!analyze -v -hang</code>	Анализ зависшего приложения (deadlock/I/O/loop)

Вспомогательные команды

Команда	Описание
<code>!eeversion</code>	Версия CLR/CoreCLR
<code>!m</code>	Список загруженных модулей (DLL)
<code>k / kbn</code>	Native стек вызовов
<code>s -u <range> "string"</code>	Поиск Unicode строки в памяти

Интерактив

Вопросы



если есть
вопросы



если вопросов нет

Подведём итоги занятия

Теперь вы сможете:

Анализировать дампы из production

Понимать .net на уровне «как оно работает внутри»

Диагностировать «невидимые» проблемы

Понимать, где искать причины падения приложения

Чтобы не бояться production



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

